Workflow di progetto

Dallo schematico all'ordine JLCPCB

VERA-HDMI-Expansion

RetroBitLab

4 novembre 2026

Strumenti: KiCad 5 (Eeschema, CvPcb, Pcbnew) per schematico e PCB, la libreria `~/Progetti/RetroBitLab-Library` per simboli, footprint e modelli 3D, e gli script del repository per tutto ciò che si produce (PDF, Gerber, 3D, BOM, CPL).

In questa fase **non** si esportano a mano PDF, BOM, Gerber o CPL da KiCad: li genera tutti `output_job.sh` al passo 10, sempre dai sorgenti.

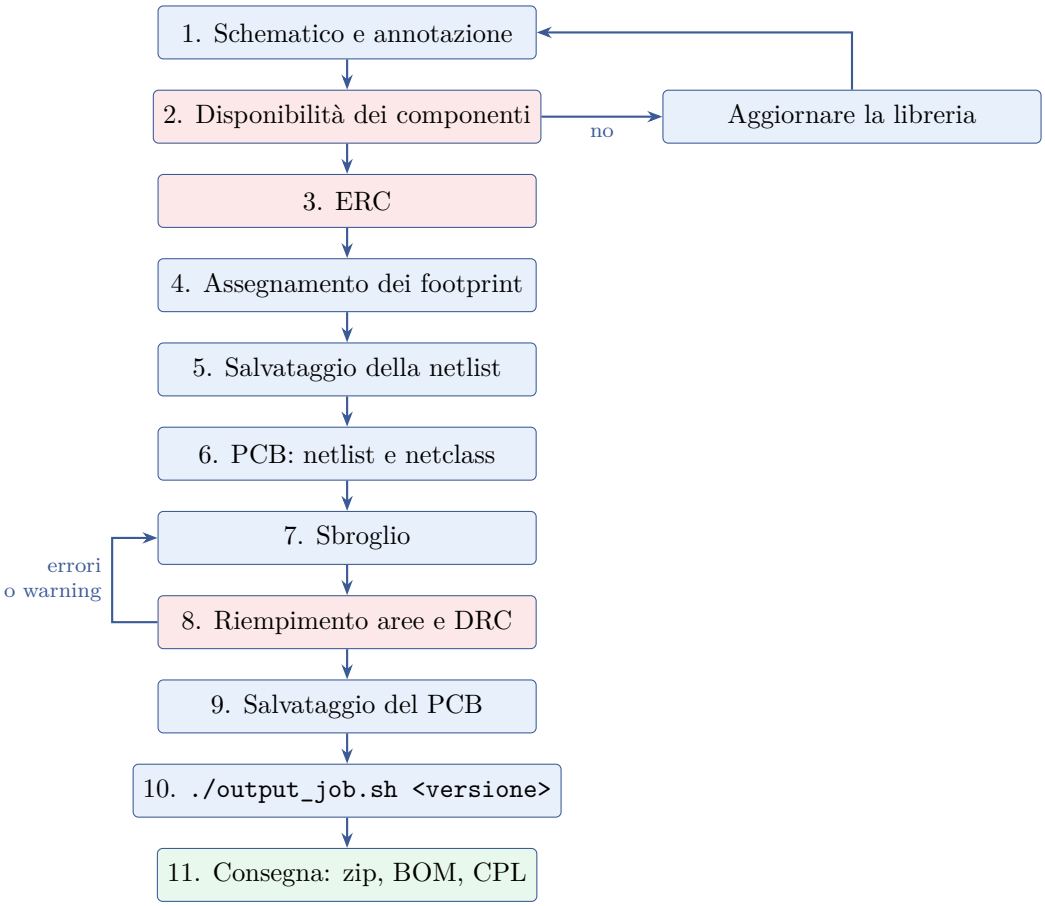


Figura 1: Sequenza dei passi. In rosso i controlli che possono rimandare indietro.

Indice

1 Schematico e annotazione **3**

2 Disponibilità dei componenti **3**

3 ERC **4**

4 Assegnamento dei footprint **4**

5 Salvataggio della netlist **4**

6 PCB: netlist e netclass **4**

7 Sbroglio **4**

8 Riempimento delle aree e DRC **5**

9 Salvataggio del PCB **5**

10 Generazione dei file di produzione **5**

11 Consegna **6**

1. Schematico e annotazione

- Si disegna lo schematico in Eeschema (KiCad 5), foglio unico `VERA-HDMI-Expansion.sch`.
- Ogni componente da montare ha il campo **`LCSC`** con il codice del pezzo (es. `C17713`). La BOM raggruppa per valore, footprint e LCSC; un componente senza LCSC non viene montato da JLCPCB.
- Si annota tutto (*Strumenti* → *Annota schematico*): nessun riferimento deve restare con `?`.

2. Disponibilità dei componenti

Subito dopo l'annotazione, prima di andare avanti:

```
./check_jlcpcb_stock.sh           # 5 schede
./check_jlcpcb_stock.sh -n 10     # 10 schede
```

Lo script costruisce la BOM dallo schematico in una cartella temporanea (XML con `kicadcli`, non scrive file del progetto) e la confronta con il catalogo JLCPCB in tempo reale. Per ogni riga riporta stock, quantità necessaria (con scarto e minimo), tipo Basic / Preferred / Extended e costo.

Stato	Significato	Blocca
`OK`	stock sufficiente	no
`LOW`	sufficiente ora, ma meno di 5 volte il necessario	no (avviso)
`GLOBAL`	stock insufficiente, ma il codice è in `jlcpcb-global-sourcing.txt`	no
`SHORT`	stock minore del necessario	sì
`OUT`	stock zero	sì
`MISSING`	codice non presente nel catalogo JLCPCB	sì

Per le righe che bloccano propone alternative con stock sufficiente (stessa categoria e package, stesso valore per i passivi, Basic prima). Esce con 0 se tutto si può montare, 1 se qualcosa va cambiato, 2 se il servizio non risponde.

Se un componente è attivo ma JLCPCB ne ha pochi, lo si può far comprare a JLCPCB con il **Global Sourcing**: si aggiunge il codice LCSC a `jlcpcb-global-sourcing.txt` (uno per riga, `#` per il commento) e all'ordine si sceglie Global Sourcing per quella riga.

Aggiornare la libreria, se necessario

Quando un componente va sostituito:

1. in `RetroBitLab-Library` si crea o si aggiorna il simbolo (`.lib`) con footprint, datasheet e campo `LCSC`, e la sua documentazione (`.dcm`); per i passivi si usa la libreria della famiglia giusta (`CCAP0805`, `CRES0805`, ...). Se cambia il package servono anche footprint e modello 3D;
2. si verifica sul datasheet piedinatura e land pattern rispetto al footprint;
3. si fa il commit della libreria;
4. nello schematico si sostituisce il simbolo e si torna al passo 2 finché il controllo non passa.

3. ERC

Ispeziona → *Controllo regole elettriche*. Si correggono tutti gli errori e i warning prima di proseguire.

4. Assegnamento dei footprint

Con CvPcb ogni simbolo deve avere il suo footprint. Per le parti JLCPCB il footprint deve corrispondere al package del codice LCSC.

5. Salvataggio della netlist

Si genera e si salva la netlist `VERA-HDMI-Expansion.net` (*Strumenti* → *Genera netlist*, formato Pcbnew). **Nessun PDF e nessuna BOM** in questa fase.

6. PCB: netlist e netclass

- In Pcbnew (KiCad 5) si importa la netlist (*Strumenti* → *Carica netlist*).
- Le **netclass si definiscono e si assegnano nel PCB** (*File* → *Impostazioni scheda* → *Classi di rete*): con KiCad 5 non esiste back annotation dal PCB allo schematico, quindi l'unico posto dove le netclass sono definite è il `.kicad_pcb`. Dopo ogni nuovo import della netlist si controlla che le net nuove o rinominate siano nella netclass giusta. Netclass della scheda: `Default`, `100R` (coppie differenziali, 0,2 mm con distanza 0,2 mm) e `POWER` (0,5 mm).

7. Sbroglio

Scheda a 4 strati: F.Cu e B.Cu per i segnali, **In1.Cu misto** (aree di 3V3, 5V e delle alimentazioni del convertitore, ma anche piste di segnale), **In2.Cu piano GND**.

A mano, oppure con fastroute per completare una scheda già sbrogliata. fastroute va installato nel sistema dal suo pacchetto Debian, da `https://github.com/gianlucarenzi/fastroute`:

```
scripts/build-deb.sh
sudo apt install ./dist/fastroute_*.deb
```

Gli script sono gli stessi di VERA-MODULE-RBL-XE e -C64 e ricavano da soli scheda e strati (`tools/board_info.py`).

Per i **collegamenti mancanti** si usa `tools/route_incremental.sh WORKDIR` (KiCad chiuso, DSN esportato con *File* → *Esporta* → *Specctra DSN* dalla scheda salvata con le aree riempite): le piste esistenti restano fisse e il risultato è `VERA-HDMI-Expansion.ses`, da importare con *File* → *Importa* → *Sessione Specctra*. Nella prima fase gli strati interni sono piani puri; con `INNER_FALLBACK=1 tools/route_incremental.sh WORKDIR` una seconda fase li usa anche per i segnali, a costo 25 volte quello degli strati esterni: su questa scheda In1.Cu porta già dei segnali, quindi la seconda fase è spesso utile. Una pista su un piano lo taglia: dopo l'import si controlla nel DRC che non restino pad isolati. Due cose da sapere:

- KiCad 5 esporta i segmenti consecutivi come un'unica polilinea e mette le giunzioni a T sui vertici intermedi o a metà dei segmenti; fastroute collega le piste solo agli estremi e vedrebbe interrotte net già complete. `tools/split_dsn_wires.py` spezza le piste nei punti di giunzione.

- L'import della sessione in KiCad 5 **sostituisce tutte** le piste e le via: `tools/filter_ses.py` tiene lo sbroglio esistente una sola volta e con la larghezza originale, aggiunge il nuovo solo sulle net incomplete (`tools/unrouted_nets.py`) e rifiuta una sessione a cui manca qualcosa.

Per far ottimizzare a fastroute (lunghezza delle piste, numero di via) solo lo sbroglio aggiunto dopo un commit, lasciando fisso quello già validato, si usa `tools/optimize_new.sh COMMIT WORKDIR`. Prima dell'import della sessione si tolgono le teardrop, dopo si rimettono. Il report di fastroute è più severo del DRC di KiCad (controlla anche il rame della stessa net e vede via ed estremità delle piste come ottagoni): il riferimento resta il DRC.

Le **coppie differenziali** (netclass `100R`) si sbrogliano a mano: gli script non le trattano come coppie e le lasciano fisse se sono già sbrogliate.

8. Riempimento delle aree e DRC

1. Al termine dello sbroglio si riempiono tutte le aree (tasto `B`).
2. Si lancia il DRC (*Ispeziona* → *Controllo regole di progettazione*).
3. Si correggono **tutti** gli errori e i warning, poi si ripetono i punti 1 e 2 finché il DRC è pulito.

9. Salvataggio del PCB

Il `.kicad_pcb` si salva sempre corretto (DRC pulito) e **con le aree riempite**: Gerber, 3D e CPL vengono generati dal file così come è salvato.

`REFILL_ZONES=1 ./output_job.sh <versione>` riempie di nuovo le aree solo in memoria, senza modificare il file: è un ripiego, non sostituisce il passo 8.

10. Generazione dei file di produzione

```
./check_jlcpcb_stock.sh          # di nuovo, lo stock cambia nel tempo
./output_job.sh <versione>       # es. ./output_job.sh 1.2
```

`output_job.sh`, in ordine:

1. XML della netlist dallo schematico con `kicadcli` → `VERA-HDMI-Expansion.xml`;
2. BOM raggruppata con `xsltproc` e `/.kicad/bom/bom2grouped_csv_jlcpcb.xsl` → `VERA-HDMI-Expansion.csv`;
3. **BOM JLCPCB** (`jlcpcb-check-bom.py`) → `<NOME>-v<versione>-JLCPCB-BOM.csv`;
4. **CPL JLCPCB** dal PCB (`run_generate_cpl.sh`, `clean_virtual.sh`) → `<NOME>-v<versione>-JLCPCB-CPL.csv`;
5. **Gerber e forature** con Pcbnew 5 e le impostazioni di stampa salvate nel PCB (`generate_gerbers.py`) → `production/` (svuotata prima);
6. **BOM interattiva** con il plugin InteractiveHtmlBom e le impostazioni di `ibom.config.ini` (`generate_ibom.py`) → `BOM/ibom.html`, quella pubblicata su GitHub Pages;
7. **PDF dello schematico** con eeplot → `VERA-HDMI-Expansion.pdf`;
8. **scheda in 3D**: STEP con kicad2step (o kicad-cli), VRML con FreeCAD → `images/VERA-HDMI-Expansion.step` e `images/VERA-HDMI-Expansion.wrl`;

9. zip del contenuto di `production/` → `<NOME>-v<versione>.zip`.

`<NOME>` è `VERA-HDMI-Expansion` per questo progetto.

11. Consegna

Il progetto da mandare in produzione è composto da **3 file**:

File	Contenuto	Dove si carica su JLCPCB
`<NOME>-v<versione>.zip`	Gerber e forature	ordine PCB
`<NOME>-v<versione>-JLCPCB-BOM.csv`	BOM	PCB Assembly → BOM
`<NOME>-v<versione>-JLCPCB-CPL.csv`	posizioni e rotazioni	PCB Assembly → CPL

All'ordine:

- nell'anteprima di montaggio si controlla la rotazione dei componenti, soprattutto quelli nuovi o con footprint cambiato;
- per le righe elencate come `GLOBAL` dal controllo stock si sceglie Global Sourcing.

Zip, BOM e CPL versionati e `production/` non vanno nel repository; ci vanno invece i sorgenti (schematico, netlist, PCB), `VERA-HDMI-Expansion.xml` e `.csv`, il PDF, la BOM interattiva e i modelli 3D.